AIAA 3102 (2026-27 Fall)

Python 编程与人工智能

深度学习

Chengwei QIN@AI Thrust

今日提纲

深度学习与 PyTorch

- 为何需要深度学习框架

PyTorch 中的张量

- 属性、创建、变形、将张量移至 GPU
- 张量运算：逐元素、广播、原地、索引 / 切片、拼接、堆叠、拆分、聚合、线性代数

自动求导与模型构建

- 计算图、损失函数、反向传播与自动求导、优化器
- 用 nn.Module 构建模型
- model.train(), model.eval(), torch.no_grad()
- 保存与加载模型

Dataset 与 DataLoader

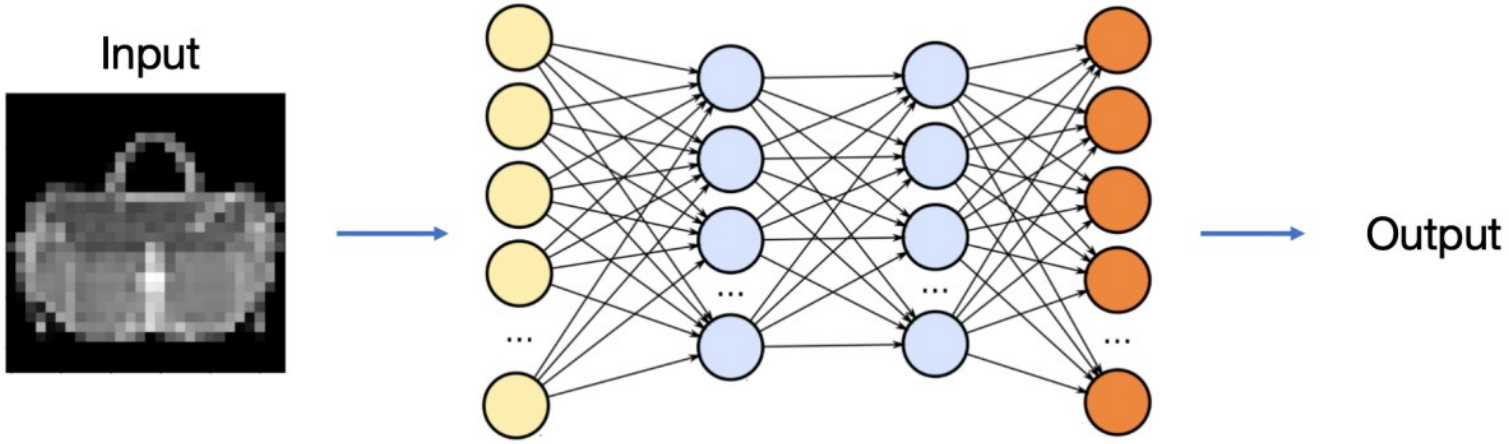
- 内置数据集、自定义数据集



深度学习与 PyTorch

什么是深度学习?

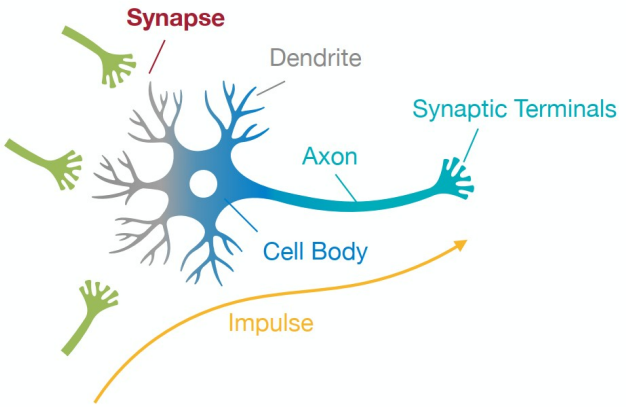
机器学习的一个子集，基于人工神经网络（ANN），并通过表示学习工作。

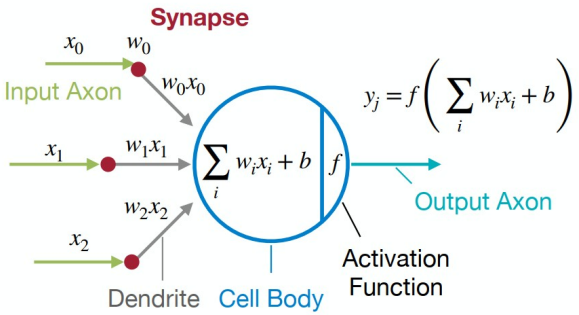


人工神经网络（ANN）

受动物大脑中生物神经网络结构与功能启发的机器学习模型。

- 单元 / 结点：模拟神经元。
- 边 / 权重：模拟突触。





信息沿边传递，经聚合后再变换

$$z = \sigma(\mathbf{w}^T\mathbf{x} + b)$$

深度学习为何如此强大?

深度学习在建模复杂数据与任务方面具有出色的表达能力。

万能逼近定理（UAT）：
至少含一个隐藏层且使用非线性激活函数的 ANN，可以任意精度逼近任意连续函数。

深度学习构成了现代人工智能应用的基础。

深度学习架构示例

模型因隐藏单元的连接方式不同而有所差异。

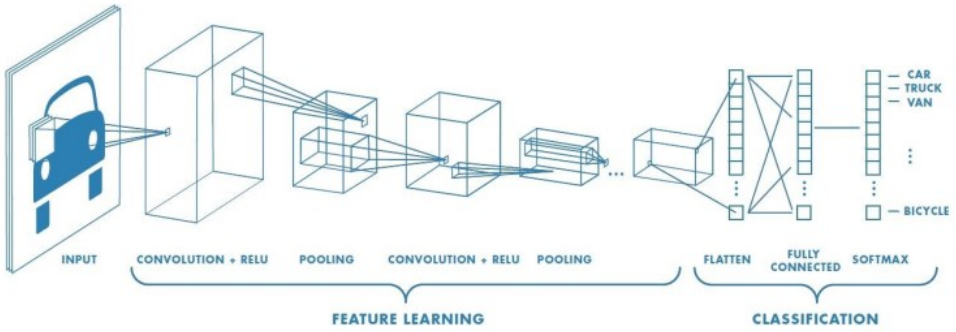


Figure: CNN for image classification

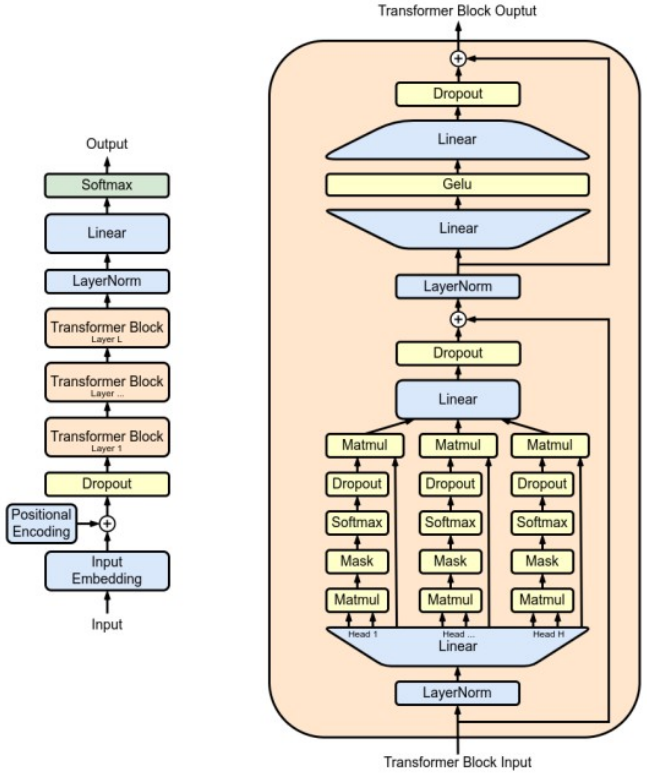


Figure: Transformer for texts (e.g., GPT)

为何需要深度学习框架

构建深度学习模型需要：

- 定制网络结构（单元与边）。
- 使数据适配网络输入。
- 用合适的优化器训练模型。
- （模型可能非常大）——加速海量计算。

深度学习框架

深度学习框架是提供工具、协助开发者构建与训练深度学习模型的库：

- GPU 支持
- 自动微分（Autograd）
- 内置且经过优化的网络层与损失函数



为何选择 PyTorch？

尽管还有许多其他框架（TensorFlow、Keras），PyTorch 在科研与教学中最为流行。

- 快速的 GPU 支持。
- 接口与 NumPy 相似。
- 内置算法、模型与数据集构成丰富的生态。
- 灵活的 nn.Module，便于封装模型。
- 动态计算图（Autograd）：易于编写与调试。
- 庞大的社区与丰富的第三方扩展库（如 Hugging Face）





PyTorch 中的张量

什么是 PyTorch 张量?

torch.Tensor 是 PyTorch 中的基本运算对象。

GPU 上的 NumPy： PyTorch 张量是多维数组，与 NumPy ndarray 高度相似

- NumPy ndarray:
  - 仅在 CPU 上运行。
  - 不跟踪运算过程。
- PyTorch 张量：
  - 可在 CPU 或 GPU 上运行。
  - 可自动构建图以计算梯度（Autograd）。

导入与创建张量

标准导入方式为 import torch

```python
import torch

# Create a 1D tensor (a vector) from 0 to 11
x = torch.arange(12)
print(x)
```

Output

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

创建张量：初始化器

创建具有特定值的张量，例如全零或全一

```
# A tensor of all zeros
z = torch.zeros((2, 3, 4))
print("Zeros shape:", z.shape)

# A tensor of all ones
o = torch.ones((3, 4))
print("Ones:\n", o)
```

用随机值初始化模型参数。

```
# Random numbers from a standard normal distribution
# (mean=0, std=1)
r_norm = torch.randn(3, 4)
print("randn:\n", r_norm)

# Random numbers from a uniform distribution [0, 1)
r_unif = torch.rand(2, 2)
print("\nrand:\n", r_unif)
```

创建张量：从 Python 列表与 NumPy 数组

可以直接从 Python 列表或 NumPy 数组创建张量

```
# Create from a nested list
T_list = torch.tensor([[2, 1, 4, 3],
                       [1, 2, 3, 4],
                       [4, 3, 2, 1]])
print(T_list)

# Create from a NumPy array
import numpy as np
A = np.array([[1, 2], [3, 4]])
T_np = torch.tensor(A)
print(T_np)
```

与 NumPy 互通

在 PyTorch 与 NumPy 之间转换很容易，且它们**共享同一块底层内存（在 CPU 上）**。

```
# To NumPy
a_torch = torch.ones(5)
b_numpy = a_torch.numpy()
print(f"NumPy array: {b_numpy}")

# From NumPy
a_numpy = np.ones(5)
b_torch = torch.from_numpy(a_numpy)
print(f"PyTorch tensor: {b_torch}")

# They share memory!
a_torch[0] = 99
print(f"NumPy array changed: {b_numpy}")
```

Output

```
NumPy array: [1. 1. 1. 1. 1.]
PyTorch tensor: tensor([1., 1., 1., 1., 1.], dtype=torch.float64)
NumPy array changed: [99.  1.  1.  1.  1.]
```

张量属性： .shape 与 .numel()

可以查看张量的属性

```
x = torch.arange(12)

# .shape tells us the size of each dimension
print(x.shape)

# .numel() tells us the total number of elements
print(x.numel())
```

Output

```
torch.Size([12])
12
```

改变形状： .reshape()

只要元素总数不变，就可以改变张量的形状。

```
x = torch.arange(12)

# Reshape x into a 3x4 matrix (2D Tensor)
X = x.reshape(3, 4)
print(X)
```

Output

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

用 -1 灵活变形

可以使用 -1 让 PyTorch 自动推断某一维的大小。

```
x = torch.arange(12)

# Automatically infer the number of rows
X1 = x.reshape(-1, 4)
print("X1 shape:", X1.shape)

# Automatically infer the number of columns
X2 = x.reshape(3, -1)
print("X2 shape:", X2.shape)
```

Output

```
X1 shape: torch.Size([3, 4])
X2 shape: torch.Size([3, 4])
```

张量属性： dtype 与 device

张量具有数据类型，并存储在某一设备上（CPU 或 GPU）。

```
tensor = torch.rand(3, 4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")
```

Output

```
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

将张量移至 GPU

为进行快速计算，可将数据与模型移至 GPU 。

```
# Check if a GPU is available
if torch.cuda.is_available():
    # 1. Define the device
    device = torch.device("cuda")

    # 2. Create a tensor on the CPU (default)
    x_cpu = torch.rand(2, 2)
    print("x on CPU:\n", x_cpu)

    # 3. Move the tensor to the GPU
    x_gpu = x_cpu.to(device)
    print("\nx on GPU:\n", x_gpu)

    # Operations on x_gpu will now be executed on the GPU
else:
    print("No GPU available, running on CPU.")
```

张量运算：逐元素

这些运算对每个元素独立作用。

```
x = torch.tensor([1.0, 2, 4, 8])
y = torch.tensor([2, 2, 2, 2])

print(x + y)
print(x * y)
print(x ** y) # Exponentiation
```

张量运算：广播

PyTorch 会自动扩展大小为 1 的维度以匹配形状

```
a = torch.arange(3).reshape((3, 1)) # Shape [3, 1]
b = torch.arange(2).reshape((1, 2)) # Shape [1, 2]

# a (shape 3,1) and b (shape 1,2)
# a is stretched 2x along dim 1
# b is stretched 3x along dim 0
c = a + b
print(c)
print(c.shape)
```

Output

```
tensor([[0, 1],
        [1, 2],
        [2, 3]])
torch.Size([3, 2])
```

原地运算

普通运算会创建新张量；原地运算则修改已有张量。

```
X = torch.rand(2, 2)
Y = torch.rand(2, 2)

# Standard (out-of-place)
id_before = id(X)
X = X + Y
id_after = id(X)
print("New tensor created:", id_before == id_after)

# In-place (more efficient)
id_before = id(X)
X += Y
id_after = id(X)
print("In-place operation:", id_before == id_after)
```

Output

```
New tensor created: False
In-place operation: True
```

张量运算：索引与切片

索引与切片的用法与 NumPy 相同

```
X = torch.arange(12).reshape((3, 4))
print("Original:\n", X)
# Get row 0
print("Row 0:", X[0])
# Get column 1
print("Column 1:", X[:, 1])
# Get a single element
print("Element [1, 2]:", X[1, 2])
# Set a slice
X[0:2, 0:2] = 99
print("\nModified:\n", X)
```

```
Original:
 tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
Row 0: tensor([0, 1, 2, 3])
Column 1: tensor([1, 5, 9])
Element [1, 2]: tensor(6)
```

```
Modified:
 tensor([[99, 99,  2,  3],
        [99, 99,  6,  7],
        [ 8,  9, 10, 11]])
```

张量运算：拼接

可以用 torch.cat() 将张量拼接在一起

```
X = torch.arange(6).reshape((2, 3))
Y = torch.ones((2, 3))

# Concatenate along dim=0 (rows)
cat_0 = torch.cat((X, Y), dim=0)
print("Cat dim=0 shape:", cat_0.shape)

# Concatenate along dim=1 (columns)
cat_1 = torch.cat((X, Y), dim=1)
print("Cat dim=1 shape:", cat_1.shape)
```

Output

```
Cat dim=0 shape: torch.Size([4, 3])
Cat dim=1 shape: torch.Size([2, 6])
```

张量运算：堆叠

torch.stack 沿新维度将一组张量连接起来。
序列中的所有张量必须具有相同形状。

```python
import torch
a = torch.rand(2, 3)
b = torch.rand(2, 3)

# Stack along new dim=1
c = torch.stack((a, b), dim=1)
print(c.shape)
```

Output

```
torch.Size([2, 2, 3])
```

- a.shape 为 [2, 3]
- b.shape 为 [2, 3]
- 堆叠 2 个张量。
- 在 dim=1 处新增一维。
- 结果形状： [2, 2, 3]

张量运算：拆分

torch.split 沿某一维将张量拆成更小的块。
可以指定每一块的大小。

```python
import torch
a = torch.rand(4, 6, 10)

# Split along dim=1 into
# chunks of size 2
lst = a.split(split_size=2, dim=1)

print(len(lst), lst[0].shape)
```

Output

```
3 torch.Size([4, 2, 10])
```

- 输入形状： [4, 6, 10]
- 沿 dim=1 拆分（大小为 6）
- 每块大小为 2
- 6 / 2 = 3 块
- len(lst) 为 3
- 每块形状： [4, 2, 10]

张量运算：聚合（sum 、 mean 、 max ）

沿某一维对张量做“聚合”。

```
a = torch.rand(4, 6, 10)

# Sum all elements
print("Total sum:", a.sum())

# Sum along a specific dimension (dim=1)
# Shape changes from [4, 6, 10] -> [4, 10]
print("Sum(dim=1) shape:", a.sum(dim=1).shape)

# Get the mean and max value
print("Mean:", a.mean(), "Max:", a.max())
```

Output

```
Total sum: tensor(112.8433)
Sum(dim=1) shape: torch.Size([4, 10])
Mean: tensor(0.4702) Max: tensor(0.9843)
```

线性代数： torch.mv 、 torch.mm 、 @

做线性代数运算时，使用专用函数。

```
A = torch.rand(3, 4)
B = torch.rand(4, 5)
x = torch.rand(4)

# Matrix-Vector Product
mv_prod = torch.mv(A, x) # Shape [3]

# Matrix-Matrix Product
mm_prod = torch.mm(A, B) # Shape [3, 5]

# The @ operator is a convenient shortcut!
mm_prod_at = A @ B
print(torch.allclose(mm_prod, mm_prod_at))
```



自动求导与模型构建

为何需要 Autograd？梯度的必要性

- 我们的目标是训练模型，即找到最佳参数（权重与偏置）。
- 我们用**梯度下降**来最小化损失函数。
- 为此，必须计算损失对每个模型参数的梯度

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \ldots$$

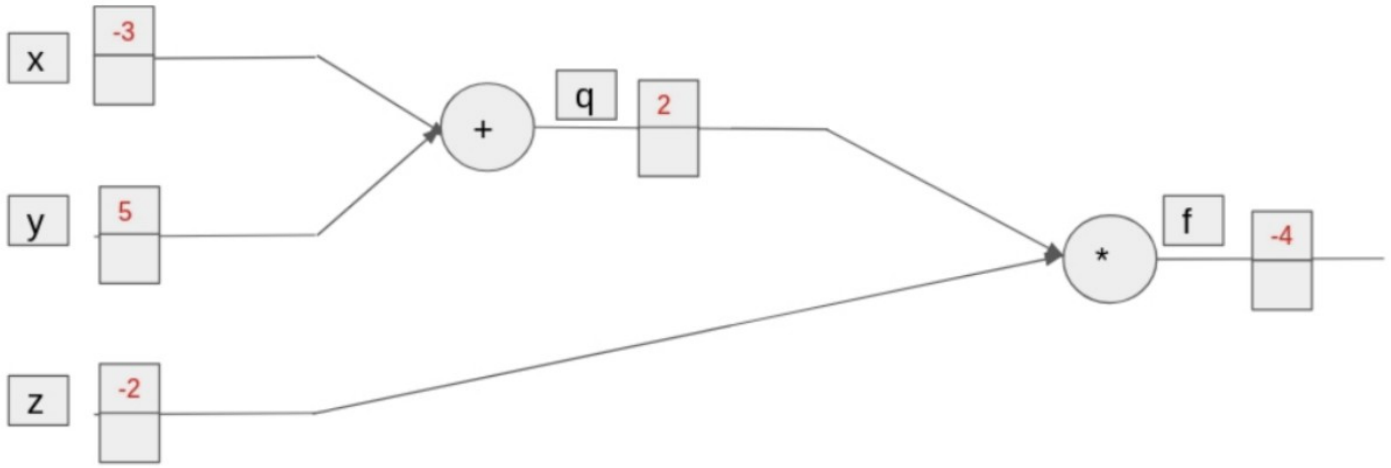
挑战： 现代深度神经网络可能有数百万甚至数十亿参数。用手写链式法则推导梯度公式几乎不可行，极其复杂且易出错。

解决方案：

- Autograd （自动微分）系统会自动为我们计算所有这些梯度。
- 为此，它首先需要构建一张“计算图”来记录所有运算。

计算图

- 张量运算会构成计算图
- **前向传播**：输入数据沿隐藏层向前传递，直至得到输出。
  - PyTorch 会跟踪对张量及其输入执行的所有运算，并存储在计算图中。



示例： $q = x + y$， $f = q * z$

用张量运算构建神经网络

可以用张量运算为神经网络构建计算图。

```
import torch
X = torch.rand(10)
w1 = torch.rand(10, 20)
w2 = torch.rand(20, 20)
w3 = torch.rand(20, 4)

h1 = torch.tanh(X @ w1)
h2 = torch.tanh(h1 @ w2)
y_pred = torch.softmax(h2 @ w3, dim=0)
y_pred
```

示例： 4 层网络 $[10, 20, 20, 4]$

在合适的参数下，万能逼近定理表明该网络可以非常有表达力。

问题：如何找到这些参数？

深度神经网络参数学习概览

- 指定损失函数
- 通过梯度下降进行优化
  - 反向传播
  - PyTorch 自动求导
- 指定优化器

损失函数

- 衡量预测偏离目标值的程度。
- 这是模型训练中需要优化（最小化）的量。
- 示例：均方误差（ MSE ）

$$\min_{\theta} \frac{1}{n} \sum_{i}^{n} (y_i - f_\theta(x_i))^2$$

- 示例：交叉熵损失

$$\min_{\theta} -\frac{1}{n} \sum_{i}^{n} [y_i \log(f_\theta(x_i)) + (1 - y_i) \log(1 - f_\theta(x_i))]$$

损失函数

- PyTorch 提供多种内置损失函数。
- **工作方式**：它们接收两个张量（如预测与目标），计算逐元素损失，再聚合成一个标量。

nn.L1Loss	nn.GaussianNLLLoss	nn.HingeEmbeddingLoss	nn.MultiLabelSoftMarginLoss
nn.MSELoss	nn.KLDivLoss	nn.MultiLabelMarginLoss	nn.CosineEmbeddingLoss
nn.CrossEntropyLoss	nn.BCELoss	nn.HuberLoss	nn.MultiMarginLoss
nn.CTCLoss	nn.BCEWithLogitsLoss	nn.SmoothL1Loss	nn.TripletMarginLoss
nn.NLLLoss	nn.MarginRankingLoss	nn.SoftMarginLoss	nn.TripletMarginWithDistanceLoss

```
from torch.nn import CrossEntropyLoss
y_true = torch.tensor(1)

# Example y_pred
# tensor([0.1694, 0.4777, 0.0183, 0.3346])

loss_f = CrossEntropyLoss()
loss = loss_f(y_pred, y_true)
```

通过梯度下降进行优化

- **优化**：寻找使损失函数最小的参数的过程。
- **梯度下降**：沿负梯度方向更新模型参数。
- 关键在于得到损失对所有参数的梯度（即偏导数）。

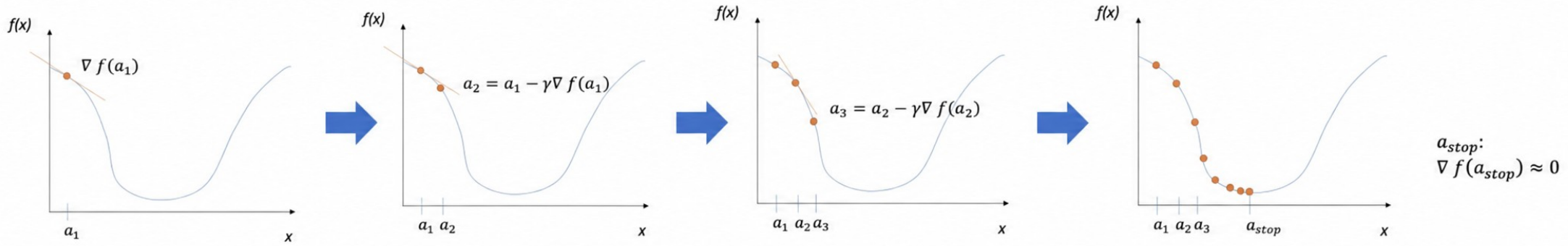


Figure: Gradient descent iteratively updates parameters to find a minimum.

反向传播

- **反向传播：**损失的梯度沿计算图反向传播，图中每个变量 / 参数都会得到其梯度

Sum:

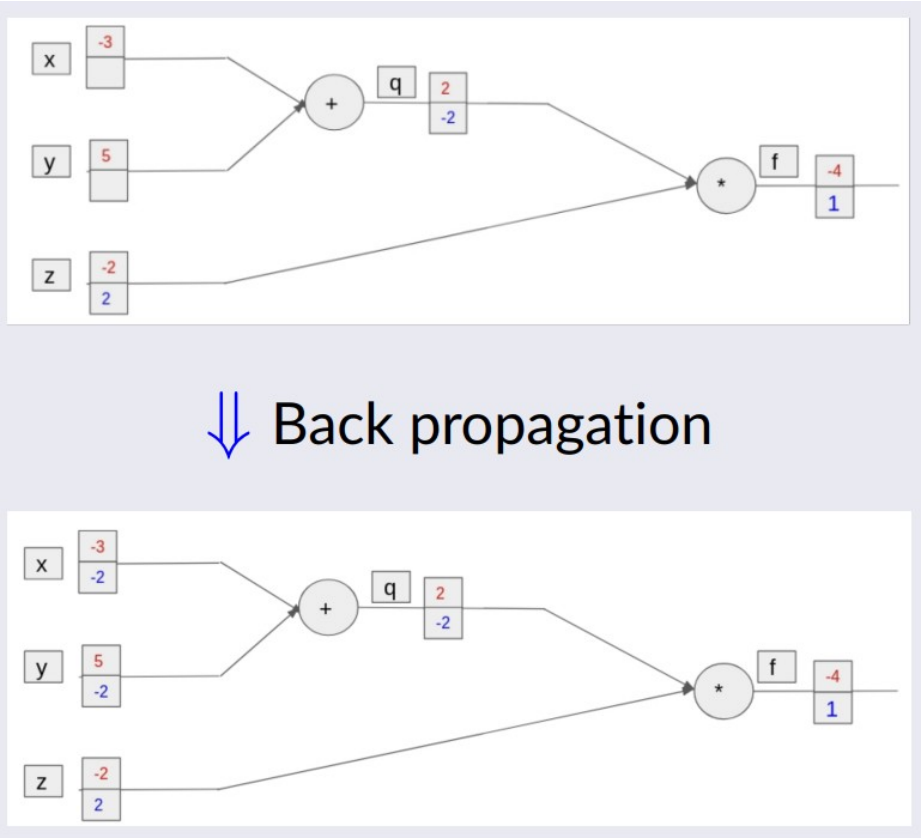
$$\frac{d}{dx}(f(x)+g(x)) = \frac{d}{dx}f(x) + \frac{d}{dx}g(x)$$

Product rule:

$$\frac{d}{dx}(f(x)g(x)) = f(x)\frac{d}{dx}g(x) + g(x)\frac{d}{dx}f(x)$$

Chain rule:

$$\frac{d}{dx}f(g(x)) = \frac{df}{dg}\cdot\frac{dg}{dx}$$



示例： q = x + y ， f = q ∗ z

PyTorch Autograd： 自动微分

- backward() 执行自动微分。
- 它沿计算图反向传播梯度，并将梯度存入每个带有 requires_grad=True 的张量的 .grad 属性中。

```
import torch
x = torch.tensor(-3., requires_grad=True)
y = torch.tensor(5., requires_grad=True)
z = torch.tensor(-2., requires_grad=True)
q = x + y
f = q * z
f.backward() # This computes all gradients

print("Gradient of z is: " + str(z.grad))
print("Gradient of y is: " + str(y.grad))
print("Gradient of x is: " + str(x.grad))
```

```
Gradient of z is: tensor(2.)
Gradient of y is: tensor(-2.)
Gradient of x is: tensor(-2.)
```

对神经网络参数使用 Autograd

- 我们为模型权重设置 requires_grad=True 。
- 调用 loss.backward() 将计算它们的全部梯度。

```
import torch
from torch.nn import CrossEntropyLoss
X = torch.rand(10)
y_true = torch.tensor(1)
# Set requires_grad=True for weights
w1 = torch.rand(10, 20, requires_grad=True)
w2 = torch.rand(20, 20, requires_grad=True)
w3 = torch.rand(20, 4, requires_grad=True)

# (Forward pass...)
h1 = torch.tanh(X @ w1)
h2 = torch.tanh(h1 @ w2)
y_pred = torch.softmax(h2 @ w3, dim=0)
loss_f = CrossEntropyLoss()
loss = loss_f(y_pred, y_true)
loss.backward() # Compute gradients

# Gradients are now populated
# w1.grad, w2.grad, w3.grad
```

对神经网络参数使用 Autograd

- 更标准的做法是使用 nn.Parameter 。

```
from torch import nn
# nn.Parameter automatically
# sets requires_grad=True and
# signals this is a model parameter
#
w1 = nn.Parameter(torch.rand(10, 20))
w2 = nn.Parameter(torch.rand(20, 20))
w3 = nn.Parameter(torch.rand(20, 4))

# ...
```

优化器

- torch.optim 是包含多种优化算法（SGD、Adam、RMSprop 等）的包。
- 示例之一：随机梯度下降（SGD）
  - 我们初始化一个优化器，向其传入待更新参数（params）以及学习率（lr）γ。
  - 更新规则（以 SGD 为例）：$\theta_t \leftarrow \theta_{t-1} - \gamma g_t$
  - optimizer.step() 使用各参数的 .grad 属性更新参数。

```
from torch.optim import SGD

# Assume w1, w2, w3 are parameters
# (from previous slide)
# loss.backward() has been called

# 1. Initialize the optimizer
optimizer = SGD([w1, w2, w3], lr=0.01)

# 2. Tell the optimizer to update the parameters
optimizer.step()

# w1, w2, w3 have now been modified
```

整合起来：训练循环

训练循环在多次迭代中重复这些步骤。

```
# (Imports and w1, w2, w3 setup)
optimizer = SGD([w1, w2, w3], lr=0.01)
n_steps = 5

for step in range(n_steps):
    # 1. Forward pass
    h1 = torch.tanh(X @ w1)
    h2 = torch.tanh(h1 @ w2)
    y_pred = torch.softmax(h2 @ w3, dim=0)

    # 2. Calculate loss
    loss_f = CrossEntropyLoss()
    loss = loss_f(y_pred, y_true)
    # 3. Reset gradients
    optimizer.zero_grad()
    # 4. Backward pass (Autograd)
    loss.backward()
    # 5. Update parameters
    optimizer.step()
```

核心模式

所有 PyTorch 训练的核心模式

```
# Inside your loop...

# 1. Reset gradients
optimizer.zero_grad()

# 2. Calculate loss and compute new gradients
loss = ...
loss.backward()

# 3. Update the model's parameters
optimizer.step()
```

必须调用 optimizer.zero_grad()，因为梯度默认会累加。

PyTorch 使用动态计算图，因此每次前向传播都会重新创建图。

随堂练习

目标是构建、训练并评估一个学习预定义函数的神经网络。

- 步骤 1：生成数据。由下列函数生成数据集（例如 100 个样本）：

$$y = 3x_1 + 4x_2 - x_3^2$$

- 步骤 2：构建模型。构建根据 x 预测 y 的神经网络。需要自行设定深度（层数）与宽度（神经元数）。
- 步骤 3：训练模型。在数据上训练网络。监控训练损失并观察其随时间的变化。
- 步骤 4：评估模型。生成新的测试数据并在其上评估模型表现。

注意：

- 输入张量 X 的形状应为 $(100, 3)$。
- 目标张量 y 的形状应为 $(100,)$。
- 损失函数（如 nn.MSELoss）可直接作用于张量，而不仅是标量。



用 nn.Module 构建模型

直接使用原始张量的问题

管理原始权重张量列表（‘w1’、‘w2’、‘w3’...）既杂乱又易出错。

- 必须把 ‘[w1, w2, w3]’ 传给优化器。
- 必须在主循环里手写前向逻辑 ‘h1 = torch.tanh(X @ w1)’。
- 这不是可复用、自包含的“模型”。

解决方案： **nn.Module**

- PyTorch 提供基类 nn.Module，帮助我们更清晰地组织模型。

Torch 模块： nn.Module 类

- 面向神经网络的**高层抽象**，提供易用接口以定义并训练复杂模型。
- 它将权重（参数）与前向运算打包进一个统一的模块。

优点：

- 模块化：可以嵌套（模块套模块）。
- 可复用：定义一次，多次使用。
- 已优化：包含常见结构的高效实现。
- 易于 GPU 加速：只需调用 .to(device) 。

Torch 模块： nn.Module 类

两类：

- 自定义模块（我们将学习如何构建）
- 内置模块（如 nn.Linear 、 nn.Conv2d ）

定制复杂模块

我们创建一个继承自 nn.Module 的类。

```
import torch.nn as nn
import torch.nn.functional as F # Contains activation functions

class MySimpleNet(nn.Module):

    # 1. Define layers in __init__
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__() # Always call this first!

        # Define the layers. PyTorch will automatically
        # register these as parameters to be trained.
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.layer2 = nn.Linear(hidden_size, output_size)

    # 2. Define the forward pass logic
    def forward(self, x):
        # x -> layer1 -> relu -> layer2
        x = self.layer1(x)
        x = F.relu(x)
        x = self.layer2(x)
        return x
```

定制复杂模块

现在使用该模块既清晰又简单。

```
# 1. Instantiate the model
model = MySimpleNet(input_size=10, hidden_size=20, output_size=2)
model.to(device) # Move all parameters to the GPU

# 2. Create an optimizer that *automatically* finds all parameters
# We just pass in model.parameters()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 3. Create dummy data
inputs = torch.rand(64, 10).to(device) # A batch of 64
labels = torch.randint(0, 2, (64,)).to(device)

# --- The training loop is now much cleaner! ---
optimizer.zero_grad()
predictions = model(inputs) # Calls the forward() method
loss = nn.CrossEntropyLoss()(predictions, labels)
loss.backward()
optimizer.step()

print("Training step complete!")
```

nn.Module 的魔力

当我们继承 nn.Module 并把一层赋为属性（例如 self.layer1 = nn.Linear(...)）时，PyTorch 会：

- 自动**注册该层**，并将其权重与偏置加入模型的参数列表。
- 这就是 model.parameters() 可以直接使用的原因。
- 这也是 model.to(device) 能把所有已注册子模块及其参数移到目标设备的原因。

替代方案： nn.Sequential

对于简单的前馈模型， nn.Sequential 是一种无需手写类即可快速构建模型的方式。

```
input_size = 10
hidden_size = 20
output_size = 2

model_seq = nn.Sequential(
    nn.Linear(input_size, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, output_size)
)

model_seq.to(device)
print(model_seq)
```

```
Sequential(
  (0): Linear(in_features=10, out_features=20, bias=True)
  (1): ReLU()
  (2): Linear(in_features=20, out_features=2, bias=True)
)
```

内置模块

PyTorch 几乎为一切都提供了优化好的预构建模块。

Module Type	Examples
Linear	`nn.Linear`
Convolutional	`nn.Conv2d`, `nn.Conv1d`
Recurrent	`nn.RNN`, `nn.LSTM`, `nn.GRU`
Transformer	`nn.Transformer`, `nn.TransformerEncoderLayer`
Activation	`nn.ReLU`, `nn.Sigmoid`, `nn.Tanh`, `nn.Softmax`
Pooling	`nn.MaxPool2d`, `nn.AvgPool2d`, `nn.AdaptiveAvgPool2d`
Normalization	`nn.BatchNorm2d`, `nn.LayerNorm`
Dropout	`nn.Dropout`

model.train() 与 model.eval()

某些模块（如 Dropout 与 BatchNorm ）在训练与评估时行为不同。

- model.train(): (Default) Tells all modules (like Dropout) to be in “training” mode. We should call this before your training loop.
- model.eval(): Tells all modules to be in “evaluation” mode. Dropout is turned off, BatchNorm uses running statistics. We must call this before evaluating our model on the test set.

torch.no_grad()

做评估时，无需构建计算图或计算梯度。

用 with torch.no_grad(): 包裹评估代码可以：

- 禁用梯度计算。
- 大幅节省内存
- 加快计算。

```
model.eval() # Set model to evaluation mode
with torch.no_grad(): # Turn off autograd
    for inputs, labels in test_loader:
        predictions = model(inputs)
        # ... calculate accuracy ...
```

保存与加载模型

我们不希望每次都重新训练模型。

推荐做法是只保存已学习的参数（“状态字典”）

```
# --- SAVING ---
# Assume 'model' is our trained model object
torch.save(model.state_dict(), 'model_weights.pth')


# --- LOADING ---
# 1. First, we must re-create the model's structure
model_new = MySimpleNet(10, 20, 2)

# 2. Then, load the saved weights into the structure
model_new.load_state_dict(torch.load('model_weights.pth'))
model_new.eval()
```

替代方案：保存整个模型

作为替代，也可以保存整个模型对象。

这样会同时保存模型结构与权重。

- 优点：非常简单。
- 缺点：不够灵活。保存的对象依赖于原始类定义与文件结构

This command saves the entire model object.

```
torch.save(model, 'model.pth')
```

This command loads the entire model back into memory.

```
model = torch.load('model.pth')
```



Dataset 与 DataLoader

加载数据时的问题

- 无法一次把 1TB 数据集全部载入内存。
- 需要能从磁盘按小批量加载数据。
- 每个 epoch 都需要打乱数据。
- 可能用多个 CPU 核并行加载数据，以免 GPU 空等。

解决方案： Dataset 与 DataLoader

- PyTorch 提供两个优雅的类来解决这一问题。

Dataset 类

- 用于存储与预处理数据样本的类。
- 它提供访问与操作数据的接口。

Dataset 类

内置数据集

- torchvision （计算机视觉）
- torchtext （自然语言处理）
- torchaudio （音频）

```
from torchvision import datasets
from torchvision.transforms import ToTensor
train_dataset = datasets.FashionMNIST(root='data', download=True,
    train=True, transform=ToTensor())
test_dataset = datasets.FashionMNIST(root='data', download=True, train
    =False, transform=ToTensor())
```

Dataset 类

自定义数据集

- 继承 torch.utils.data.Dataset
- 自定义数据集通常定义：
  - __init__(self, ...)： 加载或准备数据（例如读取 CSV ）
  - __len__(self)： 返回数据集大小
  - __getitem__(self, idx)： 返回第 idx 个样本

自定义数据集示例

泰坦尼克号数据集

- 以 Pclass、 Fare、 Age、 SibSp、 Parch 为特征， Survived 为标签；
- 用最后 100 个样本作为测试集

```
from torch.utils.data import DataLoader, Dataset
import pandas as pd
import torch
class TitanicDataset(Dataset):
    def __init__(self, path, train=True):
        super().__init__()
        titanic_df = pd.read_csv(path).fillna(0)
        if train:
            self.data = titanic_df.iloc[:-100]
        else:
            self.data = titanic_df.iloc[-100:]
        self.X = self.data[['Pclass', 'Fare', 'Age', 'SibSp', 'Parch'
            ]].values
        self.y = self.data['Survived'].tolist()
    def __len__(self):
        return len(self.y)
    def __getitem__(self, index):
        return torch.tensor(self.X[index], dtype=torch.float32), \
               torch.tensor(self.y[index], dtype=torch.float32)
train_dataset = TitanicDataset('data/titanic.csv', train=True)
train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=
    True)
```

DataLoader 类

它包装 Dataset，以实现高效、灵活的数据加载。

- 主要用途：遍历数据集，返回批量数据。
- 它替我们处理复杂操作：
  - 从数据集中采样
  - 将数据整理成一个 batch
  - 预取 batch 以加速
  - 并行加载数据（num_workers）
  - 打乱数据
  - 支持分布式训练

使用 DataLoader

创建 DataLoader：传入 Dataset 对象并设定 batch 相关参数

```
from torch.utils.data import DataLoader

# (train_dataset and test_dataset from previous slide)

train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=
    True, num_workers=10)
test_dataloader = DataLoader(test_dataset, batch_size=64, shuffle=
    False, num_workers=10)
```

遍历各个 batch

```
for X, y in test_dataloader:
    print(X.shape, y.shape)
```

```
torch.Size([64, 1, 28, 28]) torch.Size([64])
torch.Size([64, 1, 28, 28]) torch.Size([64])
...
```

随堂练习

- 创建一个能处理泰坦尼克号数据集中 5 个数值特征（Pclass、Fare、Age、SibSp、Parch）并预测是否生还的神经网络。
- 用训练数据训练网络。
- 评估测试损失。

注意：最终激活函数使用 nn.Sigmoid()。

完整流程： CIFAR-10 图像分类器

让我们把所有概念合在一起，构建一个完整的图像分类器。

- Dataset： 使用 torchvision.datasets.CIFAR10 。
- DataLoader： 用 DataLoader 包装它。
- Module： 用 nn.Module 定义卷积神经网络（ CNN ）。
- Autograd： 定义损失（ nn.CrossEntropyLoss ）与优化器（ optim.SGD ）。
- 训练：运行完整训练循环。

1. 加载数据

```
import torchvision
import torchvision.transforms as transforms

# Images are 3-channel (RGB), 32x32 pixels
# We normalize them to the range [-1, 1]
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = datasets.CIFAR10(root='./data', train=True, download=True,
    transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
    shuffle=True, num_workers=2)

testset = datasets.CIFAR10(root='./data', train=False, download=True,
    transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
    shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')
```

2. 定义 CNN 模型

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        # 3 color channels, 6 output channels, 5x5 kernel
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120) # 5x5 is the image dim
            after 2 pools
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10) # 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # Flatten all dims except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
net = Net().to(device)
```

3. 定义损失与优化器

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

4. 训练网络

```
net.train() # Set model to training mode
for epoch in range(2):  # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data[0].to(device), data[1].to(device)

        # 1. zero the parameter gradients
        optimizer.zero_grad()
        # 2. forward + backward
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        # 3. optimize
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss /
                2000:.3f}')
            running_loss = 0.0
print('Finished Training')
```

5. 测试网络

```
correct = 0
total = 0
net.eval() # Set model to evaluation mode
with torch.no_grad(): # We don't need gradients for testing
    for data in testloader:
        images, labels = data[0].to(device), data[1].to(device)

        # calculate outputs by running images through the network
        outputs = net(images)

        # the class with the highest energy is what we choose as
            prediction
        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()
print(f'Accuracy of the network on the 10000 test images: {100 *
    correct // total} %')
```

回顾今日内容

深度学习与 PyTorch

- 为何需要深度学习框架

PyTorch 中的张量

- 属性、创建、变形、将张量移至 GPU
- 张量运算：逐元素、广播、原地、索引 / 切片、拼接、堆叠、拆分、聚合、线性代数

自动求导与模型构建

- 计算图、损失函数、反向传播与自动求导、优化器
- 用 nn.Module 构建模型
- model.train(), model.eval(), torch.no_grad()
- 保存与加载模型

Dataset 与 DataLoader

- 内置数据集、自定义数据集

Python 编程与人工智能

感谢聆听！